

CSED441 Intro. to Computer Vision : Assn3 Report

December 20, 2025

Student 20240505 Hyunseong Kong

1. Overview

The objective of this assignment is to design and train a CNN model from scratch for multi-class image classification into 15 categories. All experiments strictly adhere to the competition rules, which permit only CNN-based architectures and prohibit the use of any pre-trained weights. The primary goal is to maximize classification performance under constrained training conditions, while ensuring reproducibility and providing clear justification for each architectural and training-related design decision.

2. Dataset

The given dataset consists of the following components:

- Number of training images: 45,000
- Image resolution: 64×64
- Number of classes: 15

The semantic categories included in the dataset are listed below:

Furniture / Retro / Wood / Pottery / Vinyl / Styrofoam / Glass Bottle / Clothing / Bicycle / Electronics / Paper / Canes / Pet Bottle / Plastic / Fluorescent Lamp

As will be discussed in later sections, several classes exhibit relatively weak dependence on object shape alone, making the classification task more challenging. The dataset was split into training and validation subsets using an 80% / 20% ratio, which is a commonly adopted convention. To prevent class imbalance issues during evaluation, a stratified split was applied so that the class distribution remains consistent across both subsets.

3. Baseline Model

3-1. Experimental Setup

All experiments were conducted under identical training constraints to ensure fair comparison. Unless otherwise specified, the following settings were fixed across all experiments.

- Framework: PyTorch
- Loss function: Cross-Entropy Loss
- Data split: Stratified train/validation split
- Random seed: 42

3-2. Baseline Architecture

The baseline model is based on the ResNet18 architecture, which consists of stacked convolutional layers organized using residual connections. The network is composed of four stages, each containing a predefined number of residual blocks in a [2, 2, 2, 2] configuration. Each residual block follows the basic block design, which includes two consecutive 3×3 convolutional layers.

Given that the input image resolution is relatively small (64×64), aggressive early-stage spatial downsampling was avoided. Specifically, the standard 7×7 convolution and max-pooling layers commonly used in ResNet were omitted.

December 20, 2025

Instead, all convolutional layers employ 3×3 kernels with stride 1, while spatial resolution reduction between stages is achieved gradually via stride-2 convolutions.

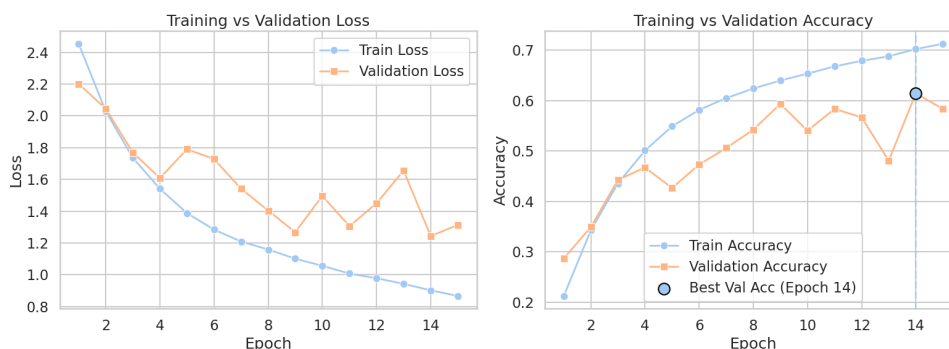
The batch size was set to 64. Although experiments confirmed that significantly larger batch sizes (up to four times larger) were feasible without stability issues, a moderate batch size was intentionally chosen. This design increases the number of parameter update steps per epoch, partially compensating for the strict limitation on the total number of training epochs.

This architecture serves as a strong and well-established baseline, enabling a fair and interpretable comparison with subsequent architectural and training strategy modifications.

3-3. Baseline Design Specification

- **Convolution blocks:** ResNet BasicBlock with two 3×3 convolution layers per block, arranged in a [2, 2, 2] stage configuration
- **Normalization:** Batch Normalization applied after each convolution layer
- **Activation function:** ReLU
- **Optimizer:** SGD with learning rate 0.1 and momentum 0.9
- **Data augmentation:** None

3-4. Baseline Performance



The baseline model achieved a maximum validation accuracy of **0.6146**.

4. Architecture Search

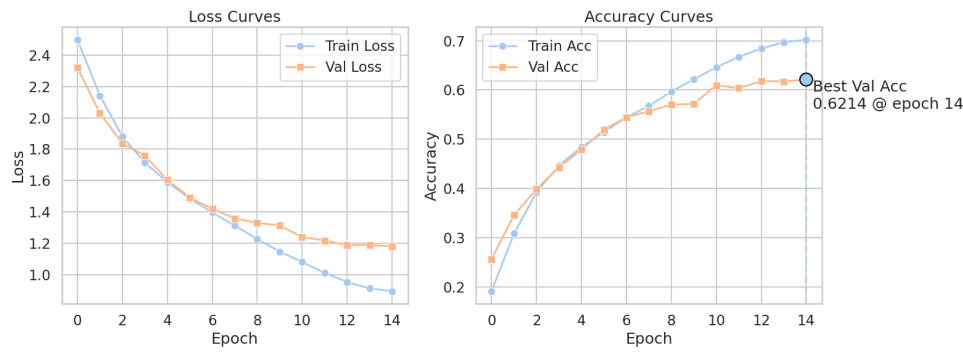
4-1. Introduction

The baseline architecture for this section is a modified ResNet18 trained from scratch. All architectural variants in this section were trained under identical conditions, including optimizer, learning rate schedule, data augmentation, and number of epochs. Only the network architecture itself was changed. Note that the performance impact is compared to the baseline model.

4-2. Architectural Variants

(a) Modification 1: ResNet34

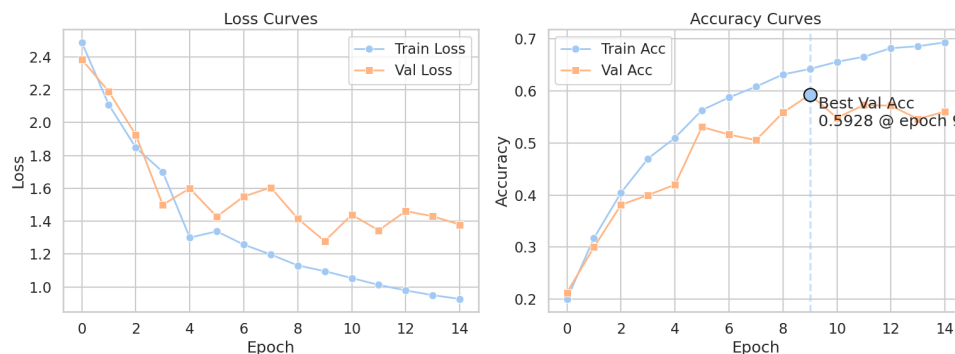
- **Design change:** The number of residual blocks per stage was increased to [3, 4, 6, 3], corresponding to the ResNet34 architecture.
- **Rationale:** Increasing network depth is a common strategy to enhance representational power and capture more complex feature hierarchies.
- **Performance impact:** 0.6214 (+0.0068)



As shown above, simply increasing the depth of the model resulted in only marginal performance gains. Although not included in the final report, additional experiments using ResNet50 yielded similar conclusions. Deeper architectures typically require a larger number of training epochs to fully converge, and under a fixed epoch budget, their potential advantages cannot be fully realized. Nevertheless, deeper models exhibited more stable training curves, suggesting improved optimization behavior. This experiment demonstrates that increasing depth alone is insufficient to achieve substantial performance improvements under the given constraints.

(b) Modification 2: Res2Net18

- **Design change:** The overall depth of the network was kept identical to ResNet18, but the backbone was replaced with Res2Net18.
- **Rationale:** Res2Net introduces channel-wise feature splitting to enable multi-scale feature representation within a single residual block.
- **Performance impact:** 0.5928 (-0.0218)



(c) Modification 3: Res2Net18SE

- **Design change:** Squeeze-and-Excitation (SE) modules were added to the Res2Net18 architecture.
- **Rationale:** While Res2Net enhances spatial multi-scale representation, SE modules recalibrate channel-wise feature responses by explicitly modeling inter-channel dependencies. The combination allows the model to emphasize informative channels derived from multi-scale features. An additional advantage is that SE blocks can be integrated without significantly altering the original architecture.
- **Performance impact:** 0.6480 (+0.0334)

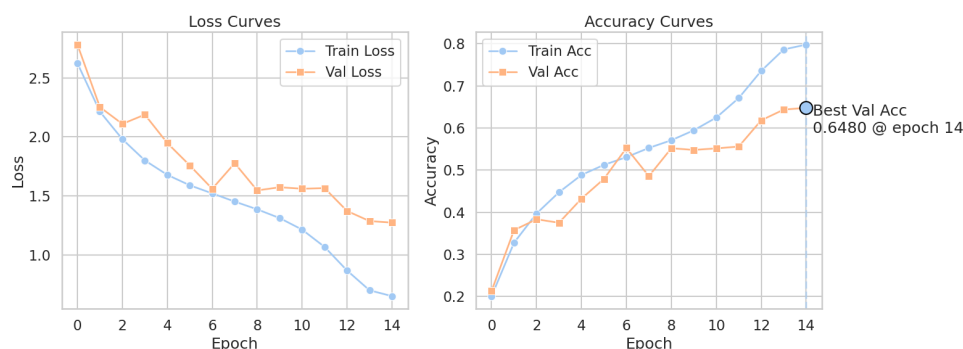


Table 1: Architecture ablation study

Model	Key change	Val Acc	Δ
ResNet18 (baseline)	Standard residual blocks	0.6146	–
ResNet34	More depth	0.6214	+0.0068
Res2Net18	Multi-scale feature representation	0.5928	-0.0218
Res2Net18 + SE	Channel attention (SE block)	0.6480	+0.0334
Res2Net18 + SE + TTA	Flip-based test-time augmentation	0.6502	+0.0356

Although Res2Net18 alone degraded performance, the introduction of SE modules led to a substantial improvement. Extensive experiments confirmed that Res2Net and SE exhibit strong synergy, justifying the retention of the Res2Net backbone despite its standalone shortcomings.

(d) Modification 4: Res2Net18SE with TTA

- **Design change:** Test-Time Augmentation (TTA) was applied during inference.
- **Rationale:** By evaluating each test image under multiple transformed views, the likelihood of misclassification is reduced, yielding an ensemble-like effect without additional training cost.
- **Performance impact:** 0.6502 (+0.0356)

Note that TTA is applied only during inference and is disabled during training. To comply with the constraint that the submission notebook must not be modified, the TTA mechanism is implemented within the model's forward function and activated only when the model is in evaluation mode.

During inference, predictions from the original image and its horizontally flipped counterpart are averaged to obtain the final logits:

$$\hat{y} = \frac{f(x) + f(\text{flip}(x))}{2}$$

This approach ensures that training dynamics remain unchanged while improving prediction robustness at test time. This can be interpreted as a deterministic ensemble for inference stabilization, as it aggregates predictions from multiple fixed transformations without introducing any additional model parameters or stochasticity during training.

4-4. Discussion & Other Architectures

Although not included in detail due to space constraints, extensive experiments were conducted using a wide range of architectures, including ResNeXt18, ResNeXt32, ConvNeXt-v1, EfficientNet-v2, DenseNet101, and MobileNet. In summary, most ResNet-family models exhibited performance comparable to or slightly worse than Res2Net-based models. ConvNeXt, however, performed significantly worse, which is attributed to its original design assumptions that target training regimes with hundreds of epochs.

Given the relatively small dataset size and the presence of substantial noise, highly expressive models tended to overfit quickly, learning dataset-specific artifacts rather than generalizable features. Consequently, stronger architectures did not consistently translate to better performance. After considering both accuracy and training efficiency, many candidate models were eliminated during the selection process. Ultimately, **Res2Net18SE with TTA** was chosen as the final model. As an additional observation, SE modules consistently provided modest performance improvements across nearly all tested architectures.

5. Data Augmentation Study

5-1. Baseline Definition

The baseline configuration for the data augmentation experiments is ResNetSE18. This is because the augmentation search was performed before identifying a better-performing model. The baseline model is trained without any data augmentation other than standard normalization. Each augmentation strategy is evaluated by adding a single augmentation component or a controlled combination of components to this fixed baseline.

5-2. Motivation

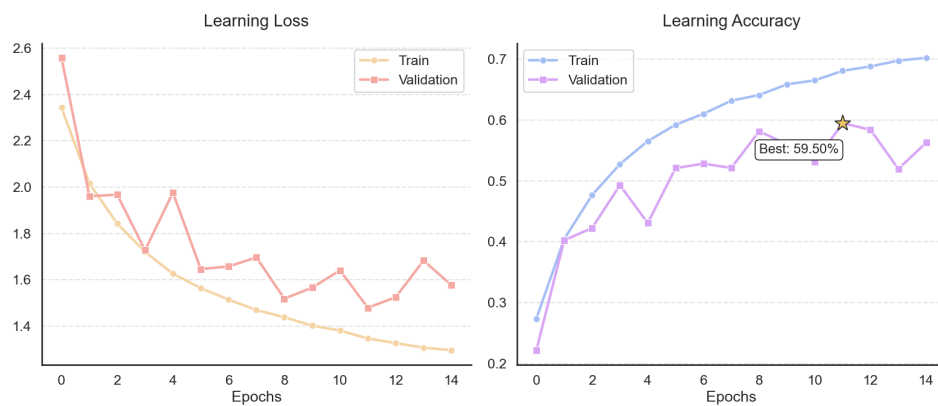
Repeated exposure to the same training samples was expected to induce overfitting and degrade generalization performance. Given the limited dataset size and the overfitting behavior observed in earlier experiments, data augmentation was introduced as a regularization strategy to improve robustness and generalization.

5-3. Augmentation Techniques

The following augmentation techniques were evaluated:

- **Random Horizontal Flip:** Horizontally flips the image
- **Random Rotation:** Applies small-angle image rotations
- **Color Jitter:** Modifies brightness, contrast, saturation, and hue
- **Random Crop:** Crops a portion of the image after padding

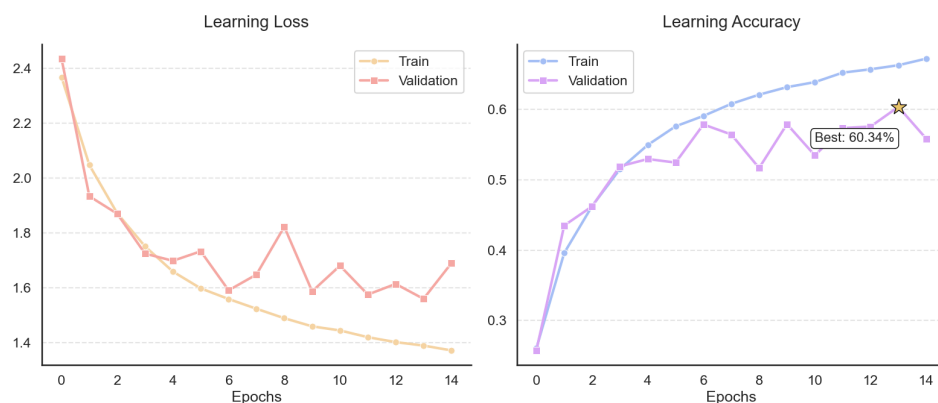
5-4. Ablation Results



The performance of the baseline model without data augmentation is shown above. The difference between this baseline performance and the previously reported Res2Net18SE results arises because these experiments were conducted prior to the final model selection and architectural optimization.

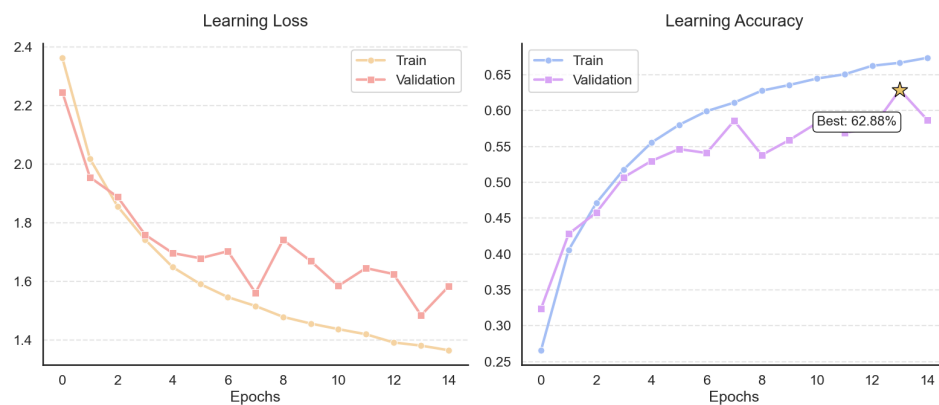
(a) Augmentation 1

- Padding 4 + RandomCrop(64)
- Performance impact: 0.6034 (+0.0082)

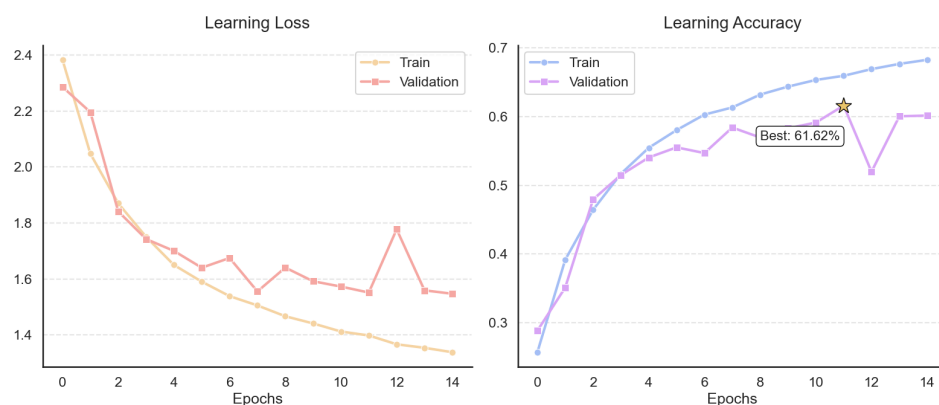


(b) Augmentation 2

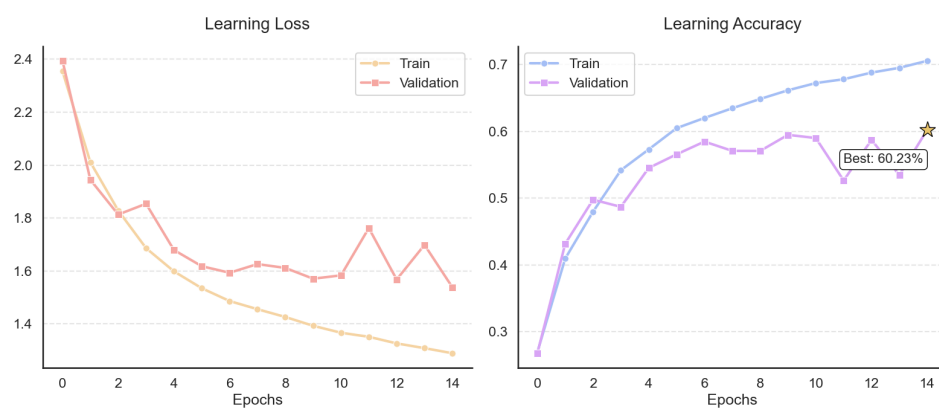
- RandomHorizontalFlip($p = 0.5$)
- Performance impact: 0.6288 (+0.0338)

**(c) Augmentation 3**

- RandomRotation(4)
- Performance impact: 0.6162 (+0.0212)

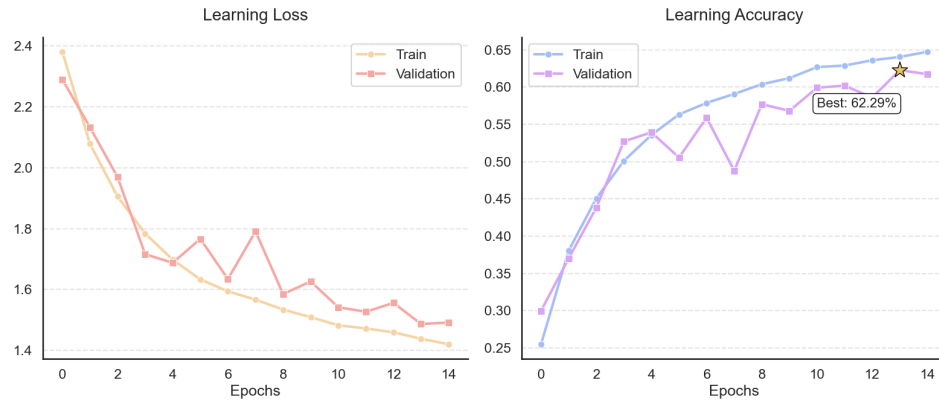
**(d) Augmentation 4**

- ColorJitter(brightness=0.02, contrast=0.02, saturation=0.01, hue=0.0)
- Performance impact: 0.6023 (+0.0073)



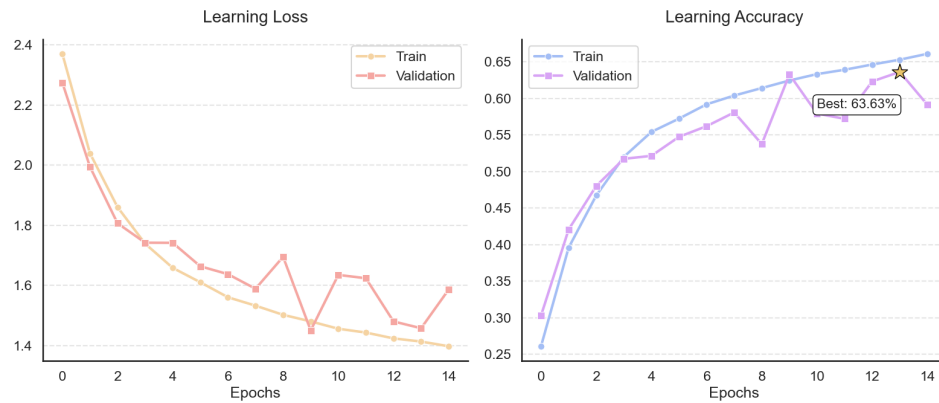
(e) Augmentation 5

- Padding 4 + RandomCrop(64)
- RandomHorizontalFlip($p = 0.5$)
- RandomRotation(4)
- Performance impact: 0.6229 (+0.0279)



(f) Augmentation 6

- Padding 4 + RandomCrop(64)
- RandomHorizontalFlip($p = 0.5$)
- RandomRotation(4)
- ColorJitter(brightness=0.02, contrast=0.02, saturation=0.01, hue=0.0)
- Performance impact: 0.6363 (+0.0413)

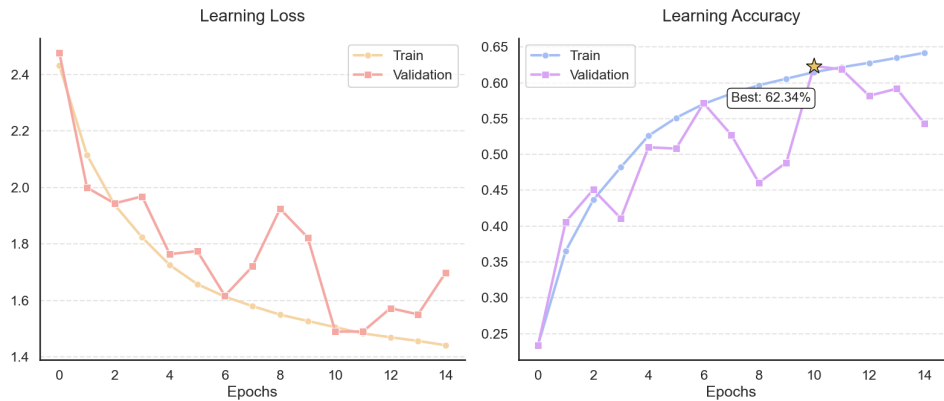


(g) Augmentation 7

- Padding 4 + RandomCrop(64)
- RandomHorizontalFlip($p = 0.5$)
- RandomRotation(4)
- ColorJitter(brightness=0.20, contrast=0.20, saturation=0.20, hue=0.10)
- Performance impact: 0.6234 (+0.0284)

Table 2: Data augmentation ablation study

Augmentation Strategy	Val Acc	Δ
None (baseline)	0.5950	–
Random Crop	0.6034	+0.0082
Horizontal Flip	0.6288	+0.0338
Rotation	0.6162	+0.0212
Color Jitter	0.6023	+0.0073
Crop + Flip + Rotation	0.6229	+0.0279
Crop + Flip + Rotation + Jitter	0.6363	+0.0413
Crop + Flip + Rotation + Jitter2	0.6234	+0.0284



5-5. Discussion

All augmentation strategies yielded some degree of performance improvement. Among single augmentations, Random Horizontal Flip produced the most significant gain, followed by Random Rotation. Color Jitter alone contributed minimally; however, as demonstrated in Augmentations (e) and (f), it exhibited a synergistic effect when combined with other transformations. Conversely, Augmentation (g) shows that overly aggressive Color Jitter negatively impacts performance.

This behavior can be attributed to the nature of several classes that rely primarily on texture, reflectance, or subtle material cues rather than object shape. Excessive augmentation distorts or removes such critical cues, hindering effective learning. Horizontal Flip, in contrast, preserves semantic and textural information while exposing the model to novel spatial configurations, explaining its superior performance.

Rotation, cropping, and jitter all introduce geometric or photometric distortions, which likely explains why their individual contributions are weaker compared to flipping. Although not included in the main report, CutMix and MixUp were also evaluated but failed to provide meaningful improvements. These techniques are known to be more effective in longer training regimes with larger epoch budgets.

Additionally, Vertical Flip was tested and resulted in degraded performance. This suggests that the validation and test datasets possess a consistent vertical orientation bias, making vertical flipping an unrealistic transformation for this task.

6. Optimizer and Training Strategy

6-1. Baseline Definition

The baseline configuration for optimizer and training strategy experiments is the best-performing model (Res2Net18SE + TTA) obtained after data augmentation tuning. This includes the selected architecture and final augmentation pipeline. In this section, the already selected final model is used as the baseline, and an ablation study is conducted by removing the added components one by one. The analysis primarily focuses on describing the impact of removing each component on model performance. All experiments in this section keep the model architecture and data augmentation fixed. Only the optimizer type, learning rate scheduler, and related hyperparameters are modified.

6-2. Training Strategy Variants

The following optimization and regularization strategies were evaluated:

- Optimizer comparison: SGD, AdamW
- Learning rate scheduling: CosineAnnealingLR, OneCycleLR
- Regularization techniques: Weight Decay, Momentum

6-3. Ablation Results

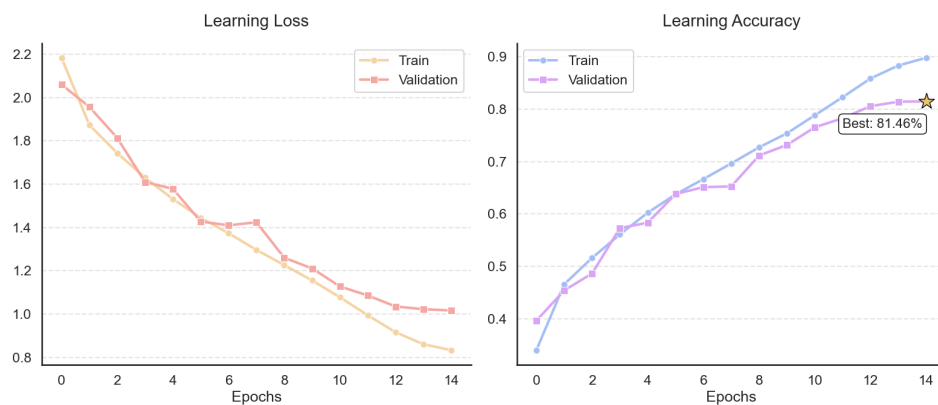


The baseline configuration shown above corresponds to Res2Net18SE trained with:

- SGD(lr=0.1, momentum=0.9, weight_decay=5e-4)
- OneCycleLR(max_lr=0.1, pct_start=0.3, div_factor=2000, final_div_factor=2000)
- Label Smoothing = 0.1

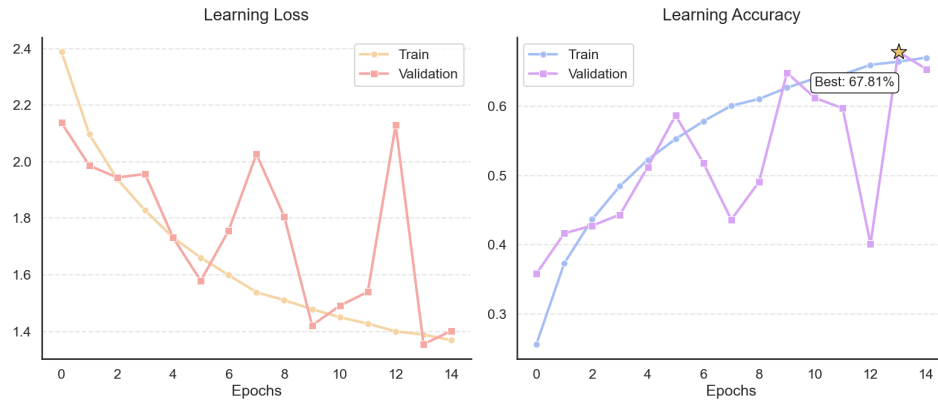
(a) Optimizer 1: AdamW

- AdamW(lr= 5×10^{-5} , weight_decay= 10^{-2})
- Performance impact: 0.8146 (+0.0063)



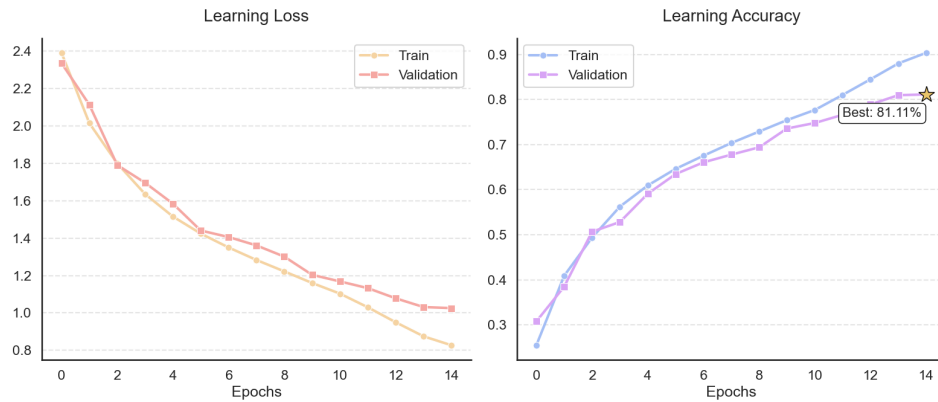
(b) Scheduler 1: CosineAnnealing

- CosineAnnealingLR($T_{\max} = 15$, $\eta_{\min} = 5 \times 10^{-5}$)
- Performance impact: 0.6781 (-0.1302)



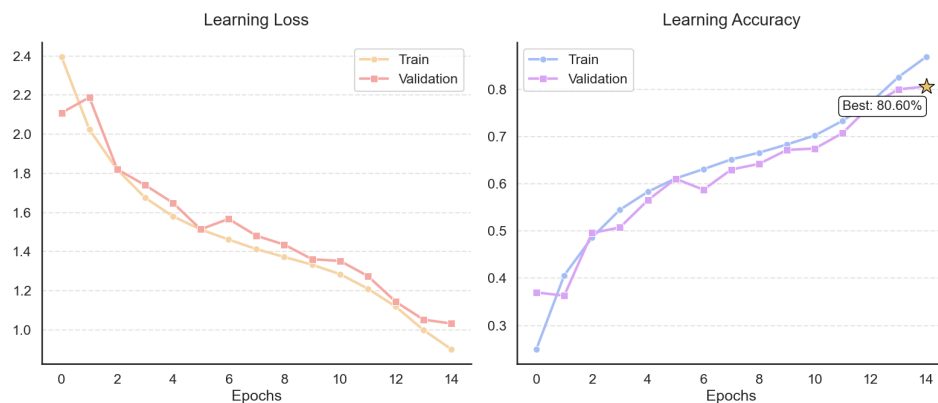
(c) Regularization 1: No Weight Decay

- No Weight Decay
- Performance impact: 0.8111 (+0.0028)



(d) Regularization 2: No Momentum

- No Momentum
- Performance impact: 0.8060 (-0.0023)



6-4. Discussion

OneCycleLR consistently outperformed CosineAnnealingLR. While cosine annealing typically provides smooth and stable convergence, the aggressive learning rate scaling required to ensure convergence within only 15 epochs led to unstable optimization. In contrast, OneCycleLR rapidly increases the learning rate from a very small initial value to a predefined maximum, forcing the optimizer to escape local minima early in training. Moreover, large learning rates inherently act as a regularizer by preventing excessive weight growth.

Table 3: Optimization and training strategy ablation study		
Training Configuration	Val Acc	Δ
SGD + Momentum + Weight Decay (baseline)	0.8083	–
AdamW Optimizer	0.8146	+0.0063
Cosine Annealing Scheduler	0.6781	-0.1302
No Weight Decay	0.8111	+0.0028
No Momentum	0.8060	-0.0023

AdamW achieved slightly better validation performance; however, SGD outperformed AdamW by approximately 1% on the test set (leaderboard score). AdamW exhibited faster loss reduction, indicating stronger overfitting. SGD, despite maintaining a higher training loss, preserved comparable accuracy, suggesting superior generalization. Regarding regularization, removing momentum degraded performance, confirming its role in stabilizing optimization trajectories. Conversely, removing weight decay slightly improved performance. Given the limited number of epochs, weights did not grow excessively, rendering weight decay more restrictive than beneficial under these conditions.

7. Final Model and Results

7-1. Final Model Configuration

The final submission model uses the following configuration:

- Architecture: Res2Net18 with SE blocks
- Data Augmentation: Random Crop, Horizontal Flip, Random Rotation, Color Jitter
- Optimizer: SGD (momentum = 0.9, weight decay = 4×10^{-5})
- Learning Rate Schedule: OneCycleLR
- Batch size: 64
- Training epochs: 15
- Label Smoothing: 0.1
- Inference: Test Time Augmentation with horizontal flip

This configuration achieves the best trade-off between validation accuracy and leaderboard performance under the given constraints.

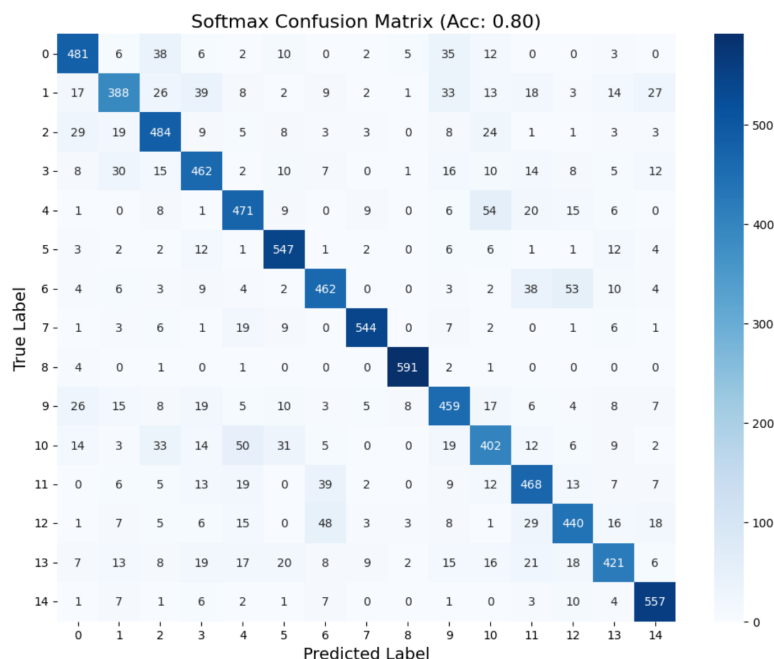
7-2. Final Performance

Final score on the leaderboard (30% public test split): 0.73333

8. Discussion and Limitations

This study demonstrates that systematic ablation across architecture, data augmentation, and optimization strategies can substantially improve CNN performance under strict training constraints. However, several limitations remain. First, the input resolution of 64×64 is extremely small, and a large portion of each image consists of background. As a result, aggressive geometric or photometric transformations often removed informative regions, leading to degraded performance. Second, the strict limit of 15 training epochs prevented deeper or more expressive architectures from fully converging. Across all models, test performance was consistently at least 2% lower than validation accuracy. In extreme cases, performance degradation reached approximately 12%, indicating severe overfitting. Furthermore, the final leaderboard score was computed on the remaining 70% of the test data not visible during evaluation, introducing additional uncertainty. Because model selection was influenced by public leaderboard feedback, human-induced selection bias cannot be ruled out.

8-1. Confusion Matrix



The confusion matrix illustrates the classes that the model predicts well and those it struggles with. Among them, Class 8, which corresponds to bicycles, is the best-performing class. In contrast, the model shows poor performance on classes such as electronic devices and materials like plastic or vinyl, whose shapes are not fixed. In particular, electronic products and retro items exhibit a wide variety of forms, and many of them are difficult to distinguish even when material information is available. As a result, the model fails to make accurate predictions for these classes.

8-2. Additional Analysis

post-hoc diagnostic only. "not used in any leaderboard submissions"

Although not used for submission, the final model was trained for 30 epochs to assess its upper-bound potential.



Under this extended training regime, validation accuracy improved to approximately 85%. This result indicates that the 15-epoch configuration is not globally optimal. More aggressive learning rate schedules or parameter search strategies could further enhance performance. While scheduler behavior changes with longer training cannot be ignored, these results suggest that additional performance gains remain feasible. The corresponding test score was not evaluated; however, given the positive correlation between validation and test performance observed throughout this study, similar improvements are expected.